CardioNary: Heart-Sound Model Roadmap

Working plan for the cardio team, Monday 26 October to Tuesday 17 November 2026

CS F407 Artificial Intelligence · Team CardioNary · Branch `cardio` · Internal working document · Revised Thu 12 Nov

1. The plan in one page

Where we are (Mon 26 Oct). Only the baseline exists: the shared code in `core/` and the leakage tests. Nothing has been committed to `cardio` yet.

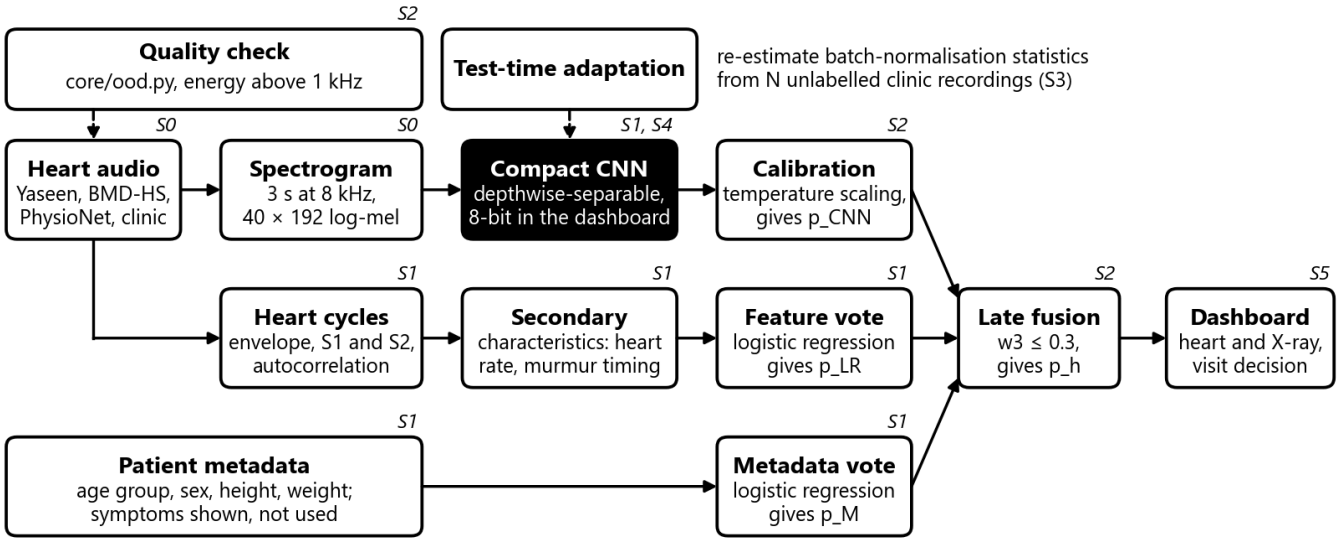
What we deliver by Tue 17 Nov. The heart-sound branch exactly as the submitted proposal describes it (Sections 4.1 to 4.7), with results, report sections, slides, AI-usage note and demo video all ready. The days after 17 Nov are buffer only.

One change from the proposal. We will not make any new recordings. The "new device" test uses only the 50 clinic recordings we already have in `Real_World_Data`: 10 patients, 5 chest positions each, 5 seconds, recorded with a MEMS microphone. They replace the "about 100 clips" in proposal 4.6, and the report says so.

Changes on Thu 12 Nov. Anshuman builds the offline tool as one Streamlit dashboard for both teams, `app/app.py`, following the frontend guide in `docs/STREAMLIT_CLINICAL_FRONTEND_GUIDE.md`. It shows the heart and X-ray results side by side and makes the visit decision (proposal 4.4), so the separate heart page is dropped. Aviral instead writes `cardio/predict.py`, the one function the dashboard calls for a heart result, and takes over `cardio/run_all.py` from Anshuman. All code, the dashboard included, ends on Mon 16 Nov; the write-ups (report and slides) run alongside it from Fri 13 to Tue 17 Nov. Sections 2 to 7 and both figures follow these changes.

Stage	Dates	Who	What we get
S0 Setup and data	Mon 26 to Tue 27 Oct	Everyone, each on their own	A working environment and your own copy of the data you need
S1 Each person's first piece	Tue 27 to Fri 30 Oct → **M1**	Everyone, nobody waits for anybody	Base CNN with its 8-bit copy; characteristics, feature vote and metadata vote; unseen datasets, metrics and noise script
S2 Calibration, late fusion, first clinic check	Mon 2 to Fri 6 Nov → **M2**	Anshuman, Shivansh, Aviral	Fusion results on Yaseen, BMD-HS and PhysioNet 2022; first look at the clinic recordings
S3 Test-time adaptation	Mon 2 to Sat 7 Nov	Shivansh, Aviral	How much adaptation recovers, including on the clinic recordings
S4 8-bit model, noise tests, edge budget	Sat 7 to Tue 10 Nov → **M3**	Aviral, Shivansh, Anshuman	8-bit model measured on a laptop; noise results
S5 Code complete, results frozen	Wed 11 to Mon 16 Nov → **M4**	Everyone	All code merged, the dashboard for both teams included; every number in a committed file
S6 Report, slides and deliverables	Fri 13 to Tue 17 Nov → **M5**	Everyone	Report sections, slides, AI-usage note and demo video, all ready; written alongside S5

M1 falls in the week of the mid-project check-in. Figure 1 shows what each stage builds, and Figure 2 shows the dates.



S0 to S6: the stage in Section 4 that builds each part. Black box: the only neural network; everything else is classical processing or a small linear model.

Figure 1. The heart-sound pipeline, as in the proposal's Figure 1. Each box is labelled with the stage that builds it.

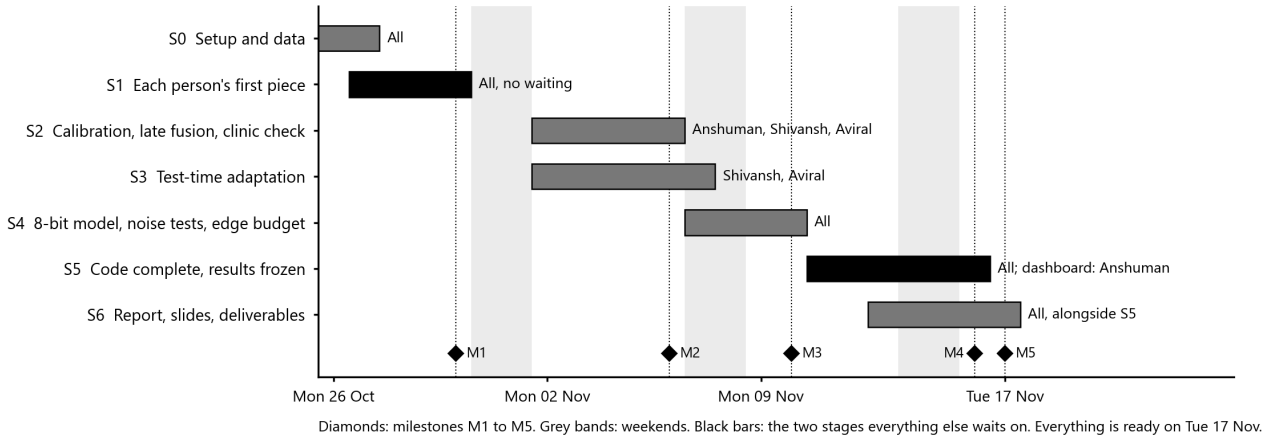


Figure 2. Dated plan. Stages S1 and S5 are on the critical path: later stages need the base CNN, and the final numbers and the demo video need frozen code. All code, the dashboard included, ends on Mon 16 Nov; the write-ups run alongside from Fri 13 Nov.

2. Choices we have made

Where the proposal left a choice, we take the option that is quickest to build and still meets the proposal's targets.

Topic	Our choice	Why
CNN (proposal 4.1)	The compact depthwise-separable CNN already in `core/train.py`, trained from scratch without augmentation	No new model code; the baseline reached 98.9% 5-fold accuracy, and its 8-bit file is 28 kB and agreed 100% with the float model
Input (4.1)	3-second windows at 8 kHz, 20 to 800 Hz, 40 × 192 log-mel spectrogram (`CARDIO_CONFIG`)	Unchanged. `load_wav` converts any sample rate to 8 kHz and takes channel 0 of a stereo file
Unseen datasets (4.6)	BMD-HS, and the public part of PhysioNet 2022	Both open and both 4 kHz, which the loader converts automatically
BMD-HS classes	Patients with one label among Normal, AS, MR, MS for the class test; every patient for the refer-or-not test	These are the four classes Yaseen shares
PhysioNet 2022 label	Patient murmur Present vs Absent; Unknown dropped	The dataset's own label, one per patient
New-device data	The 50 recordings in `Real_World_Data`, analysed without labels	No new recordings; they have no diagnosis
Patient score	Mean of the window probabilities over all of a patient's recordings	Same rule as `core/train.py`
Secondary characteristics (4.2)	`audio_descriptors` from `core/descriptors.py`, plus heart rate and regularity from the autocorrelation of the envelope	Heart-sound segmentation already exists; the autocorrelation is a few lines
Feature and metadata votes (4.2)	Logistic regression with balanced class weights; missing heights and weights filled with the training-fold median	Small, fast and easy to explain
Fusion weights (4.2)	Try every $w_1 + w_2 + w_3 = 1$ in steps of 0.05 with $w_3 \le 0.3$; keep the lowest log-loss on the training folds	A few hundred combinations, seconds to run
Calibration (4.4)	Temperature scaling with one number T	About ten lines; the model itself does not change
Adaptation (4.3)	Re-estimate the batch-normalisation statistics in one forward pass	No training, no gradients
Noise tests (4.6)	White and pink noise at SNR 20, 15, 10, 5, 0 dB; a low-pass filter at 600, 500, 400, 300, 200 Hz	NumPy and SciPy only
Symptoms	Asked and shown in the dashboard, not used by the metadata vote	No public heart-sound dataset records them, so there is nothing to learn a weight from
Offline tool (4.5)	One Streamlit dashboard, `app/app.py`, for the heart and X-ray results and the visit decision, built by Anshuman from the frontend guide; English and Hindi	One screen for the health worker; pure Python; runs without internet
Heart result for the dashboard	`cardio/predict.py`: one call takes a recording and the patient details and returns the `ModelOutput` and `FusedOutput` of `core/contracts.py`	The dashboard never touches the model or its weights, as the guide requires
Systole or diastole in the dashboard	Read from the measured systolic and diastolic energy, the rule `audio_descriptors` already uses	In v1, occlusion on the model pointed to systole for every class, Normal included (`core/descriptors.py`)
Patient data in the dashboard	Kept only in memory for the current patient (`st.session_state`) and cleared by "Next patient": the guide's option A, with no database. The one file kept is an encrypted audit log with a hashed patient code and no names	Nothing to lose if the laptop is lost; the guide recommends this for screening camps

Topic	Our choice	Why
Where code goes	Everything new goes in `cardio/`, plus new test files in `tests/`; the dashboard goes in `app/`. We only import from `core/` and never change it	The team can work without touching shared code

Left out. PhysioNet 2016, predicting murmur grade or shape, a segmentation network, the AR class, live recording in the dashboard (recordings are uploaded as WAV files), and the guide's encrypted database (its option B). Entropy minimisation is left out unless S3 finishes early.

3. Tools and data

Tool	Version	Used for
Python	3.10	Everything
TensorFlow, Keras, TensorFlow Lite	2.19.1, 3.12.4	Compact CNN, adaptation, 8-bit conversion and inference
SciPy	1.15.3	Filtering, resampling, envelope, noise tests, temperature fitting
scikit-learn	1.7.2	Logistic regressions, imputation, AUROC
NumPy, pandas	2.1.3, 2.3.3	Arrays, tables
Git LFS	any recent	BMD-HS download
Streamlit	1.65.0	The dashboard. Later releases need Python 3.11
streamlit-authenticator, fpdf2, uharfbuzz, cryptography, pydicom	0.4.2, 2.8.9, 0.56.4, 50.0.2, 3.0.2	Dashboard only: login, PDF report (uharfbuzz shapes the Hindi letters), encrypted audit log, DICOM X-rays. Pinned in `app/requirements-app.txt`
pytest, GitHub Actions	9.1.1	Tests on every pull request

Dataset	Used for	Size	Labels and metadata	Where to get it
Yaseen heart sounds	Training and 5-fold testing	1,000 clips, 200 per class, 8 kHz, 1.2 to 4 s	Normal, AS, MS, MR, MVP; no metadata	`python core/download_datasets.py --dataset yaseen`
BMD-HS (BUET)	Unseen dataset from another hospital and device	864 recordings of 20 s at 4 kHz, up to 8 positions per patient	AS, AR, MR, MS, Normal (a patient can have several), confirmed by heart ultrasound	github.com/sani002/BMD-HS-Dataset, CC BY-NC-ND 4.0
PhysioNet 2022 (CirCor DigiScope v1.0.3)	Unseen dataset for refer-or-not; trains and tests the metadata vote	The public 60%: 3,163 recordings from 942 children, 4 kHz, 450 MB	Murmur Present, Absent or Unknown; age group, sex, height, weight, pregnancy	physionet.org/content/circor-heart-sound/1.0.3, open access
`Real_World_Data`	New-device test and adaptation, as a clinic would supply	50 recordings: 10 patients × 5 positions (aortic, pulmonic, tricuspid, mitral, Erb's point), 5 s, 48 kHz	No diagnosis and no metadata	Already on our project laptop; never copied anywhere else
Noisy Yaseen copies	Controlled shift (proposal 4.6)	Made from Yaseen fold 0 (200 clips)	Same labels	Our own script

The proposal's 5,272 recordings from 1,568 children describe the whole PhysioNet 2022 dataset; only the public 60% can be downloaded.

Rules for `Real_World_Data` (from its README; these are not optional):

- Never commit it, never upload it anywhere (cloud drives, Colab, AI assistants), and never copy it to another laptop. The analysis runs on the laptop that already holds it.
- Never put a recording, a spectrogram picture or a per-patient table in the report, the demo video or the dashboard screenshots. Only totals and averages may be reported.
- Use channel 0 only, and only the 50 `p<N>_<site>.wav` files. `load_wav` already reads channel 0 of a stereo file and handles the 24-bit-in-32-bit samples.
- Inspect `p6_mitral_area.wav` before use: its README notes a handling transient.
- After each analysis, add a line to the README's status section saying what was run, so the record of how the data were used stays complete. Follow its retention rule when the work is done.

4. Stage by stage

Initials: **Av** Aviral Dwivedi, **An** Anshuman Singh, **Sh** Shivansh Singhal. Owners follow the roles in the submitted proposal; Shivansh's recording task is replaced by work on the unseen datasets.

S0 Setup and data (Mon 26 to Tue 27 Oct)

Everyone sets up alone and downloads the data they need themselves.

Everyone

☐ Set up, run the tests, and get Yaseen (it is small):

```
git checkout cardio && git pull
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt streamlit
python -m pytest tests/
python core/download_datasets.py --dataset yaseen
python -m core.build_cardio_manifest
python -m core.preprocess --dataset yaseen
```

☐ `git diff data/manifests/yaseen_manifest.csv` must show no change, which proves your copy matches the committed folds. About 10 clips are shorter than 1.5 s and give no 3-second window; that is expected.

Anshuman

☐ Also download PhysioNet 2022: the ZIP from its page, or `wget -r -N -c -np https://physionet.org/files/circor-heart-sound/1.0.3/`. Unzip into `data/raw/circor/`.

Shivansh

☐ Also download PhysioNet 2022 as above, and BMD-HS: `git lfs install`, then `git clone https://github.com/sani002/BMD-HS-Dataset data/raw/bmdhs`. A 20-second file is about 160 kB; a file of a few hundred bytes is only a Git LFS pointer, so run `git lfs pull`.

Aviral

☐ On the laptop that holds `Real_World_Data`, check that it loads: `core.features.load_wav(path, CARDIO_CONFIG)` must return 5 s of audio at 8 kHz (40,000 samples) for each of the 50 files.

Done when the tests pass for everyone, three people have the Yaseen cache, and the unseen datasets and clinic recordings load.

S1 Each person's first piece (Tue 27 to Fri 30 Oct) → M1

Every task in this stage uses only the data you downloaded in S0, so nobody waits for anybody.

Aviral: base CNN and its 8-bit copy (proposal 4.1, 4.5)

☐ Write `cardio/model/cv_oof.py`. It runs the same 5-fold loop as `core/train.py` without augmentation: copy the loop from its `main()`, which calls `run_fold` for each fold. After each fold, run the returned model on that fold's test clips, normalised with the returned mean and standard deviation, and keep the probabilities.

- Output: `reports/heart_oof_yaseen.csv` with index, patient_id, fold, label, and p_Normal, p_AS, p_MS, p_MR, p_MVP. These "out-of-fold" probabilities come from models that never saw those clips, which is what calibration and fusion need.

☐ 5-fold accuracy must be ≥ 95%; the baseline reached 98.9%. Also report macro-F1 and the recall of each class, as proposal 4.7 asks.

☐ Build the final model and its 8-bit copy, trained on folds 1 to 4 with fold 0 held out:

```
python -m core.quantize --dataset yaseen --holdout-fold 0
```

It writes the float model, the 8-bit `.tflite` file (about 28 kB) and a report with the normalisation mean and standard deviation. Put the two model files in the team's private drive folder by Fri 30 Oct; from S2 on, everyone uses this copy. Model files contain no patient data.

Anshuman: secondary characteristics, feature vote and metadata vote (proposal 4.2)

☐ `cardio/model/features.py`: for every Yaseen clip, compute:

- the numeric fields of `audio_descriptors` in `core/descriptors.py`: band powers, spectral centroid, murmur found or not, murmur length and loudness, systole and diastole length, S1 to S2 loudness ratio, click suspected, segmentation succeeded;
- heart rate and regularity: autocorrelate the envelope (the `_envelope` function in the same file) and find the strongest peak between 0.33 and 1.5 s, which covers 40 to 180 beats per minute. Heart rate = 60 / lag; regularity = the height of that peak.

Output: `reports/heart_features_yaseen.csv`. If segmentation fails, the value is left empty and later filled with the training-fold median.

☐ `cardio/fusion/votes.py`, feature vote: abnormal (AS, MS, MR or MVP) against Normal. Standardise, then logistic regression on the 5 folds already in the Yaseen manifest. Output: out-of-fold p_{LR} in `reports/heart_plr_yaseen.csv`, and its AUROC on its own.

☐ Metadata vote, using only PhysioNet 2022's `training_data.csv` (no audio needed yet):

- one row per patient; Murmur Present = 1, Absent = 0, Unknown dropped;
- inputs: age group as 0 to 4 (neonate, infant, child, adolescent, young adult), sex, height, weight, pregnancy;
- fill missing values with the training-fold median, standardise, then logistic regression;
- make 5 folds of patients yourself (`StratifiedKFold`, seed 0) and save them in `cardio/data/circor_folds.csv`, so Shivansh uses the same folds;
- output: out-of-fold p_M and its AUROC on its own.

Shivansh: unseen datasets, metrics and the noise script (proposal 4.6, 4.7)

☐ `cardio/data/build_unseen.py`: manifests and caches for BMD-HS and PhysioNet 2022.
- One row per recording: path, patient_id, labels, and metadata where present.
- Build the caches with `core.features.features_for_file(path, CARDIO_CONFIG)`.
- Commit `data/manifests/bmdhs_manifest.csv` and `data/manifests/circor_manifest.csv`; both datasets are public and anonymised.

☐ `cardio/eval/metrics.py`, with tests in `tests/test_cardio_metrics.py` on small made-up arrays:
- accuracy, macro-F1 and per-class recall;
- AUROC and sensitivity at 90% specificity;
- expected calibration error with 15 bins;
- 95% bootstrap interval from 1,000 resamples of patients, seed 0.

☐ `cardio/eval/corrupt.py`: white and pink noise at a chosen SNR, and the low-pass filter, applied to one clip.

Done when (M1, Fri 30 Oct) all three pieces work:

- the base CNN reaches ≥ 95%, its out-of-fold file exists, and the final model files are in the drive;
- the feature and metadata votes have out-of-fold results;
- the two unseen caches exist, the metric tests pass, and the noise script runs.

S2 Calibration, late fusion and a first look at the clinic recordings (Mon 2 to Fri 6 Nov) → M2

From here on, everyone uses Aviral's final model and out-of-fold file.

Shivansh (proposal 4.6, 4.7)

☐ `cardio/eval/run_unseen.py`: run the final model on BMD-HS and PhysioNet 2022, going from windows to recordings to patients. Output: `reports/heart_unseen.json` with:
- refer-or-not, with $p_{CNN} = 1 - p(\text{Normal})$: AUROC, macro-F1 at 0.5, sensitivity at 90% specificity and calibration error;
- BMD-HS classes: on single-label Normal, AS, MR and MS patients, drop the MVP probability, renormalise, and report macro-F1 and per-class recall.

☐ Run Anshuman's `features.py` on both datasets.

Anshuman (proposal 4.2, 4.4)

☐ `cardio/fusion/calibrate.py`: fit one temperature T by minimising the negative log-likelihood on log-probabilities, $z = \log p$, using the Yaseen out-of-fold file. Report calibration error before and after. Yaseen accuracy is near 99%, so $T > 1$ is expected.

☐ `cardio/fusion/late_fusion.py`: $p = w_1 \cdot p_{CNN} + w_2 \cdot p_{LR} + w_3 \cdot p_M$.
- The weights for fold k are chosen on the other folds only. A missing vote gets weight 0, and the other weights are divided by their sum.
- Return each result as `FusedOutput("heart", ...)` from `core/contracts.py`, which refuses $w_3 > 0.3$.

☐ With Shivansh, two comparisons with bootstrap intervals in `reports/heart_fusion.json`:
- **Generalisation**, using no labels from the new dataset: plain CNN against CNN plus feature vote, with weights chosen on Yaseen, on BMD-HS and PhysioNet 2022.
- **Metadata gain**, within PhysioNet 2022 folds: CNN plus feature vote, with and without the metadata vote. The weights are re-chosen on the training folds in both cases.

Aviral (proposal 4.6, new device)

☐ `cardio/eval/real_world.py`, run only on the laptop that holds the data. It writes totals and averages only, to `reports/heart_real_world.json`:
- how many of the 50 recordings pass the quality check. Use `signal_provenance` in `core/ood.py` on the 8 kHz audio converted to 16-bit. Expect many failures: the check was built to flag this kind of microphone;
- the share of recordings the model calls abnormal, overall and by chest position;
- how often the 5 positions of the same patient agree on refer-or-not;
- the share of recordings with a heart rate between 40 and 180 beats per minute, and with successful segmentation;

- these patients have no metadata, so the metadata vote is missing and its weight is spread over the other two. This is the proposal's missing-vote rule in action.

☐ Agreement between the float and 8-bit models on Yaseen fold 0 and on PhysioNet 2022.

Done when (M2, Fri 6 Nov) `reports/heart_unseen.json`, `reports/heart_fusion.json` and `reports/heart_real_world.json` exist.

S3 Test-time adaptation (Mon 2 to Sat 7 Nov)

Shivansh (proposal 4.3)

☐ `cardio/tta/bn_adapt.py`, which makes an adapted copy of the model; the original is never changed:

1. Load the float model.
2. Set its four batch-normalisation layers to `trainable = True` and `momentum = 0.0`.
3. Run the windows of the N clinic recordings through it once with `training=True`. Each layer now holds the clinic's mean and variance; no gradients are computed.
4. Set the layers back to `trainable = False` and save the adapted copy.
5. Collapse guard: if the adapted copy puts more than 95% of the N recordings in one class while the original did not, keep the original statistics.

☐ `cardio/eval/run_tta.py`:

- N = 8, 16, 32, 64 recordings, with 5 random draws each;
- shifts: BMD-HS, PhysioNet 2022, white noise at 5 dB SNR, low-pass at 300 Hz;
- the N recordings used to adapt are never scored;
- recovery = (adapted − plain) / (clean − plain), on refer-or-not macro-F1 and AUROC;
- one-sided batches: draws with 10%, 50% and 90% abnormal;
- time to adapt on a laptop CPU.

Output: `reports/heart_tta.json`.

☐ Optional, only if the above is done by Thu 5 Nov: entropy minimisation as a second method.

Aviral

☐ Adaptation on `Real_World_Data`, on the same laptop:

- adapt on patients p1 to p5 and measure p6 to p10, then swap;
- repeat the S2 measures before and after adaptation, especially how often the 5 positions of a patient agree;
- add the results to `reports/heart_real_world.json`.

S4 8-bit model, noise tests and edge budget (Sat 7 to Tue 10 Nov) → M3

☐ **Aviral:** re-quantise the adapted model, using the adaptation windows as the representative set; they need no labels. Record agreement with the adapted float model and the conversion time.

☐ **Aviral:** edge budget on one ordinary laptop, for the original and the adapted 8-bit models. Output: `reports/heart_edge_budget.json` with:

- file size;
- agreement with the float model on fold 0;
- time per recording and peak memory;
- time for one whole case: quality check, CNN, characteristics, systole-versus-diastole attribution, votes and fusion.

☐ **Shivansh:** on Yaseen fold 0, plain against adapted accuracy for:

- white and pink noise at SNR 20, 15, 10, 5 and 0 dB;
- the low-pass filter at 600, 500, 400, 300 and 200 Hz, which imitates a duller stethoscope.

Output: `reports/heart_robustness.json`.

☐ **Anshuman:** add the adapted model to the generalisation comparison, which is the full CardioNary branch.

Done when (M3, Tue 10 Nov) all three files exist and are checked against Section 5.

S5 Code complete and results frozen (Wed 11 to Mon 16 Nov) → M4

All code ends on Mon 16 Nov, the dashboard included. The write-ups in S6 start on Fri 13 Nov and run alongside.

☐ **Anshuman, by Fri 13 Nov:** the fitted numbers the dashboard needs.

- T goes in `reports/heart_calibration.json`. T, the weights, and each vote's column order, filling values, scaler and coefficients go in `reports/heart_fusion_params.json`.

- `heart_fusion(p_cnn, features, metadata)` uses only that file and returns a `FusedOutput`; p_{cnn} is the calibrated 1 − p(Normal).
- It turns age in years into the PhysioNet age group, and leaves the metadata vote out above 21 years, an age PhysioNet 2022 does not cover.

☐ **Aviral, by Fri 13 Nov:** `cardio/predict.py`, the heart result for the dashboard (the guide's Section 3):

- `analyse_heart(recording, metadata)` takes a WAV at any sample rate, as a path or from memory (`io.BytesIO`), and a `PatientMetadata`;
- quality check: `signal_provenance` from `core/ood.py`, and at least 1.5 s of sound, which gives one 3-second window. If it fails, the heart result is withheld. Failed segmentation only blanks the timing measures;
- the shipped 8-bit model on every window, then temperature T, then a `ModelOutput` whose secondary values are the 17 characteristics;
- systole or diastole from the measured energy ratios;
- votes and fusion through `heart_fusion`. Until it exists, the CNN vote counts alone with weight 1, which is the missing-vote rule;
- so that uploads stay in memory, split `characteristics(path)` in `cardio/model/features.py` into `characteristics_from_audio(x)` and the existing wrapper, with identical output; Anshuman approves that pull request;
- tests in `tests/test_cardio_predict.py` on made-up audio, so they run without data: a clean synthetic heartbeat gives valid contracts, loud broadband noise is withheld, and a 1-second clip is too short.

☐ **Aviral:** run the edge budget again through `cardio/predict.py`, so "time for one whole case" is the dashboard's own, votes and fusion included.

☐ **Aviral:** `cardio/run_all.py`, taken over from Anshuman, regenerates every `reports/heart_*.json` from the caches, in order. The clinic-recording part runs only where that data is. Run it once more on Mon 16 Nov, after the last code change, so every committed number comes from the final code.

☐ **Anshuman, by Sat 14 Nov:** the dashboard core, on the `dashboard` branch made from `develop` (see `app/README.md`):

- start from the guide's Section 7, which is a mock-up: replace `mock_heart_cnn_predict`, the fixed votes and the synthetic waveform with real calls;
- intake as a `PatientMetadata`: age in years, sex, height, weight, pregnancy, and the symptom checkboxes. Every field starts empty, and an empty field is None. The guide's sample starts at age 12, 135 cm and 32 kg, which would enter made-up values;
- heart tab: a WAV upload with playback (`st.audio`). There is no live recording, so leave out the guide's `st.audio_input`. Then, from `analyse_heart`: the quality verdict, the predicted class and calibrated p_h, each vote with its weight, the characteristics table, systole or diastole, and the spectrogram from `core/spectrogram.py`;
- X-ray tab: image upload, with the result from the lung team's `lung/predict.py` once it is in `develop`. Until then the tab says the test is not available;
- visit decision: $p_{any} = 1 - (1 - p_h)(1 - p_x)$, compared with $\tau = 1 / (1 + C_{miss}/C_{refer})$ and returned as a `VisitDecision`. A test that is missing or withheld is left out;
- English and Hindi, and no red: an amber card for referral, `[theme] primaryColor = "#0645ad"` in `.streamlit/config.toml`, never `st.error`, and text tab labels instead of the guide's heart and lung emoji, which are drawn in red;
- offline: in the same file, `[server] address = "127.0.0.1"` and `[browser] gatherUsageStats = false`, because Streamlit sends usage statistics unless told not to.

☐ **Anshuman, by Mon 16 Nov:** the rest of the dashboard, as the guide describes:

- a waveform view next to the spectrogram;
- X-rays as PNG, JPEG or DICOM (`pydicom`). Keep only the pixels, so EXIF and DICOM tags such as the patient's name never reach the page;
- the clinician's notes and override, a printable PDF report (`fpdf2`, with a Devanagari font for Hindi) and the JSON export;
- the patient code hashed with HMAC-SHA256 and a key kept on the laptop, and an append-only audit log, encrypted with `cryptography`, holding the time, the user, the hashed code and the result;
- a login (`streamlit-authenticator`) that separates triage mode from clinician mode;
- "Next patient" clears `st.session_state`; there is no database;
- tests with Streamlit's `AppTest`, which can upload a file in 1.65;
- the `dashboard` → `develop` pull request, merged on Mon 16 Nov.

☐ **Everyone:** open the `cardio` → `develop` pull request as soon as `cardio/predict.py` is in, so the dashboard gets the heart branch, and merge `cardio` into `develop` again after the last change on Mon 16 Nov.

Done when (M4, Mon 16 Nov) all code is merged into `develop`, the dashboard included; the dashboard gives a heart result for one PhysioNet 2022 recording, an X-ray result for one public image and a visit decision, with internet switched off; and `run_all.py` has regenerated every heart number from the final code.

S6 Report, slides and deliverables (Fri 13 to Tue 17 Nov) → M5

This stage runs alongside S5 from Fri 13 Nov. Write from the committed result files, and take the final numbers from them after the code freeze on Mon 16 Nov.

- ☐ Each person writes the report section and makes the slides for their part: the method in a few lines, the results table with intervals, and what did not work.
  - Aviral: 4.1, 8-bit, the clinic recordings (totals only) and the edge budget.
  - Anshuman: 4.2, calibration and the dashboard (4.5).
  - Shivansh: unseen data, adaptation and noise tests.
- ☐ **Shivansh:** check every number in the report and the slides against those files, and that each has its bootstrap interval. Check again on Tue 17 Nov, after the code freeze.
- ☐ **Shivansh:** a 2 to 3 minute demo video of the dashboard's heart tab, recorded on Tue 17 Nov once the code is frozen: offline, with a public recording, never a clinic one.
- ☐ **Everyone:** the AI-usage note (tools used, for what, how the output was checked), and experiment notes in `cardio/experiments/`, including failed ones.

Done when (M5, Tue 17 Nov) every deliverable is ready. Nothing new starts after this date.

5. Targets and what we compare

The targets are the heart rows of the proposal's success criteria (4.7), fixed in advance.

Goal	Measure	Target	Stage
Base model	Yaseen 5-fold accuracy	≥ 95%	S1
Generalisation	Refer-or-not macro-F1 gain of the full branch over the plain CNN, on BMD-HS and on PhysioNet 2022	≥ +3 points on each	S2, S4
Metadata	Referral AUROC gain from the metadata vote on PhysioNet 2022	≥ +0.02, with $w_3 \leq 0.3$	S2
Adaptation	Share of the macro-F1 lost on an unseen dataset that is recovered, N ≤ 64	≥ 50%, in under 5 s on a CPU	S3
Calibration	Expected calibration error after temperature scaling	≤ 0.05	S2
Edge budget	8-bit size; agreement with float; time per case	< 1 MB; ≥ 98%; < 1 s on a laptop CPU	S4, S5

The clinic recordings have no diagnosis, so they carry no target; their results are reported as they are.

What we compare (proposal 4.7).

- Three versions: plain CNN; CNN plus feature vote; CNN plus feature vote plus metadata vote.
- Each version with and without adaptation.
- Datasets: Yaseen (5-fold), BMD-HS, PhysioNet 2022, noisy Yaseen, and the clinic recordings, which are measured without labels.
- Metrics: accuracy, macro-F1 and per-class recall; refer-or-not AUROC; expected calibration error (15 bins); model size, memory use and time per case.
- Every result carries a 95% bootstrap interval from 1,000 resamples.

6. Each person's weeks

Member	26 Oct to 1 Nov	2 to 8 Nov	9 to 16 Nov: code	13 to 17 Nov: write-ups
Aviral	Yaseen data; base CNN; out-of-fold file; final and 8-bit models; clinic data loads	First look at the clinic recordings; adaptation on them; 8-bit agreement	Re-quantisation; edge budget; `cardio/predict.py`; `run_all.py`	Report and slides: data, CNN, 8-bit, clinic recordings, edge budget
Anshuman	Characteristics; feature vote; metadata vote	Calibration; late fusion; the two comparisons	Adaptation in the comparison; fitted numbers for the dashboard; the dashboard	Report and slides: characteristics, votes, fusion, calibration, dashboard
Shivansh	Unseen-dataset manifests and caches; metrics; noise script	Unseen-dataset runs; test-time adaptation	Noise tests	Report and slides: unseen data, adaptation, noise; checking numbers; demo video

7. How we work, rules and risks

Routine.

- A 10-minute check-in every evening.
- Make your branch from `cardio` as `cardio-<name>-<topic>`, open a pull request back into `cardio`, and have one teammate approve it. Merge with "Create a merge commit".
- If someone is unavailable, Aviral covers model runs, and Anshuman and Shivansh cover each other's analysis.

Rules that keep the numbers honest.

- Normalisation, standardisation, missing-value filling and temperature are fitted on training folds only.
- Nothing is fitted on the fold being scored.
- PhysioNet 2022 folds are split by patient.
- Yaseen has no patient IDs, so its 5-fold score is an upper bound; the unseen datasets are the honest test.
- The recordings used for adaptation are never scored.
- Every reported number comes from a script and a committed JSON file.
- Seeds are fixed at 0.

Dashboard rules.

- The dashboard fits nothing. T, the weights and the vote coefficients come from committed files.
- The HMAC key, the audit-log key and the login file are made on the laptop and never committed; only placeholders are.
- It runs offline on 127.0.0.1 and shows no red. Its screenshots and demo use public recordings only.

Risk	What we do (easiest first)
Base CNN below 95%	Check the cache and normalisation; the baseline reached 98.9% with these exact settings
BMD-HS files are only Git LFS pointers	`git lfs install` then `git lfs pull`; or download the files one by one from the GitHub page
Class sets differ between datasets	Score the shared classes only, or score at the refer-or-not level
Most clinic recordings fail the quality check	Expected for this microphone; report results for all 50 and for those that pass
Metadata vote learns a shortcut	Only patient facts go in, its weight is capped at 0.3, and recording site and campaign are never inputs; report its weight and its AUROC on its own
Feature vote adds little on 4 kHz data	Accept a small weight and report it; the characteristics still appear in the dashboard and report
Adaptation hurts on small or one-sided batches	Use N = 32 recordings by default; the collapse guard keeps the original statistics; report the failing settings
The fitted numbers for fusion are late	`cardio/predict.py` counts the CNN vote alone with weight 1, so the dashboard works from the first day; the other votes appear when `heart_fusion` lands
The X-ray result is not ready	The dashboard shows the heart result alone; a `VisitDecision` needs only one usable test
Running out of time	Drop in this order: entropy minimisation, pink noise, the BMD-HS class test (keep refer-or-not); then, in the dashboard, the login, Hindi in the PDF, and the PDF itself (keep the JSON export). Never drop freezing the results, the quality check, Hindi on the page or the no-red rule

Shared with the lung team. The dashboard joins the two branches and makes the visit decision (proposal 4.4). The lung team hands over `lung/predict.py`, its fitted numbers and matching model files through `develop` by Sat 14 Nov, and its code ends on Mon 16 Nov, as ours does. Everything else in this roadmap is the heart team's alone.